State licensing pipeline — handoff documentation

Generated 2026-07-13 from repo commit `f2555665`. To get current numbers, run `python -m pipeline.client.handoff_doc` from the repo root (add `--pdf` for a PDF).

For engineers: this document is the plain-English overview. The technical entry points are `CLAUDE.md` (repo root), `pipeline/client/README.md`, and `pipeline/config/README.md`.

1. What this is, and what you already have

This repository turns state licensing law into structured, comparable data: for each occupation and each state, what license (if any) is required, what it takes to get one, and what the license lets you do. The results are published as an interactive website.

The most important thing to know: **the raw law text is already in this repository.** Collecting statute text from the web *used to be* the most painful part of this work — it's now automated and runs unattended (see section 2) — and we have already done it broadly. Think of the full scope as a grid: every occupation we track (94) crossed with every state (50) — 4,700 occupation/state cells in all. We've gotten law text saved for 3,643 of them (78%).

That is not the same as "100% coverage is easy and we're just behind" — finding the *right* law for a given occupation and state is often genuinely hard, not a checkbox. Some occupations have their own clearly-named chapter (easy); others are a handful of sections buried inside a much broader chapter that covers many unrelated things (hard to even locate, let alone confirm complete); a few states may not license an occupation at all, which looks identical to "not found yet" until someone checks.

And it cuts both ways: even for the 78% we HAVE captured, we could have gotten it wrong without knowing it — pulled a repealed statute, a different profession's act, or a chapter that no longer matches how the law is actually organized today. A confidently-processed record on the site is not a guarantee we found everything; it's our best current attempt, not a verified-complete one. Section 7 names the specific tools built to catch exactly this class of mistake and what they've actually caught so far — worth reading if "how much can we trust this" is the real question. Treat the 78% honestly: it's how far a real, ongoing effort has gotten, not a countdown to a known, verified finish line.

Day-to-day work — turning that saved text into published data — **never touches the web-scraping machinery.** You will not need to scrape anything unless you want brand-new material or a refresh of old material.

2. Data sources & integrations — the complete inventory

Every occupation and state on the site traces back to exactly two external data sources. That's the whole list — there is no third feed, API, or vendor anywhere in the pipeline. Both are read-only, free, and require no account or contract with either source's operator.

Source	Coverage	Method	Code	Update cadence
Justia (law.justia.com)	98.6% of corpora, 49/50 states	Scrape (headless, automated)	`pull_text.py`	Annual editions

PA Legislature (legis.state.pa.us)	1.4% of corpora, 1/50 states	Scrape (plain HTTP)	`pull_text.py`	At/near enactment

(Coverage measured from the saved files' own recorded URLs, not asserted. Both sources route through the same file, `pipeline/ingest/pull_text.py` — Justia via `fetch_via_scrapling` (a headless stealth browser; a CDP-attached Chrome remains as a fallback), PA via `is_pa_legis_url` and its section parser.)

On update cadence, specifically:

- **Justia publishes each state's code as a year-labeled edition** (its own pages are literally titled things like "2024 Mississippi Code"). Several states we track (Kentucky, Oregon, Illinois, and others) publish *only* year-stamped pages — no undated "current" URL at all — and our pipeline has dedicated logic (`pull_text.py`'s `_dedupe_year_versions`) to detect this and always resolve to the latest year. That's direct, first-party evidence Justia refreshes annually, syncing to each state's newly codified text once that year's legislative session's changes are finalized — not a guess.
- **Pennsylvania's site is the state's own primary source, not a third-party aggregator**, so it isn't on a yearly sync cycle at all — a newly passed act should appear there at or near when it's officially published, well ahead of when it takes legal effect (Pennsylvania's default rule is 60 days after enactment unless the law says otherwise).
- Neither source updates in real time — the site always serves a snapshot from the last time someone ran a fetch, not a live lookup.

What needs a human, per source:

- **Justia** — its Cloudflare bot-wall is now cleared **automatically and headless** (a stealth browser, Scrapling, does it), so a fetch needs **nobody watching or clicking through anything** — no Chrome window to open, no challenge to solve by hand. (The old hand-started-Chrome method is kept only as a fallback.) If Justia changes its page layout or URL scheme, the parser needs updating.
- **PA Legislature** — there's no Cloudflare-style bot wall here, so this source has always been fetchable by a plain automated script with nobody watching. The only risk is the same as Justia's second case: if PA changes its site's URL scheme or page markup, the parser needs updating.

Fetching can now run unattended. This is a real change from what this document used to say. Re-fetching or fetching new statute text still runs on your machine, and a large batch can take a while (it's deliberately polite — a short pause between requests), but it no longer needs a person babysitting a browser: the bot-wall is cleared headless and flaky responses are retried automatically. It's much closer to "kick it off and let it run" than the old hand-driven-Chrome process.

One honest caveat — this is a cat-and-mouse solution, not a permanent guarantee. The headless fetch works by looking enough like an ordinary human browser that Cloudflare's bot-detection lets it through. That's a moving target: **if Cloudflare changes how it detects automated traffic, this could stop working** — the fetch would start coming back empty or stuck on a challenge — and it would need the stealth tool updated, or a switch back to the hand-driven-Chrome method. That older method is **deliberately kept as a fallback** for exactly this reason. So: this is the right, low-effort default *today*, and the fallback is there if the day comes that it isn't.

One more related point: **finding the law is often harder than fetching it.** Many occupations' licensing rules aren't a standalone chapter — they're a few sections buried inside a much broader one (one insurance-agent type inside the entire insurance code). The repo includes automated locator tooling for this, and the per-state mappings it produces are saved in config, so that work doesn't have to be redone.

3. What's easy, what's cheap, and what's hard

The pipeline runs in stages. Each stage takes the previous stage's output and does one job. Here is the whole path a state's law travels — from the raw web page all the way to the published site — with which steps are free and which cost money:

An interactive version of this diagram is on the site: utah-licensing.vercel.app/pipeline (linked from the home page as "How the pipeline works").

(LOCATE and CLEAN together are the "find the right acts and clean them up" work; TRIM and EXTRACT are the two AI steps that cost money; PUBLISH just packages the finished data for the site. The per-state costs shown in the diagram are approximate; the exact figures are in the cost table in section 6.)

The stages cost very different amounts, which is what the rest of this section is about. Current state of the grid, as of generation:

	Count	What it means
Published	13 occupations	Site data built and ready to serve
Extracted, not published	71 occupations	Structured data exists for at least some states; publishing is a free command
Ready to process	2,144 occupation/state cells	Law text on disk; needs only the paid AI steps (~$0.20/cell, ~$419.10 for all of them)
New occupation, not set up yet	0 cells (8 occupations)	Law text on disk, but this occupation has never been processed before — see below
Needs a fetch	13 cells	Law located but text never pulled — requires the scraping setup

The 13 published occupations: architecture, construction_trades, dentistry, dietetics, electrician, hvac, medicine, nursing, optometry, plumbing, podiatry, professional_engineering, public_accountancy.

About that "needs a fetch" number — it's a small, *known* residual, and not all of it is fresh scraping. Some of these cells are ones where the automated locator pointed at the wrong chapter and we need to re-aim it at the right one; others are occupations whose law is a handful of sections buried inside a much broader chapter that has to be scoped down first. Neither is a simple "go fetch it." Just as important, this is **not a fixed finish line**: as we keep checking the data against reality, we occasionally find an additional chapter or occupation we didn't know we needed

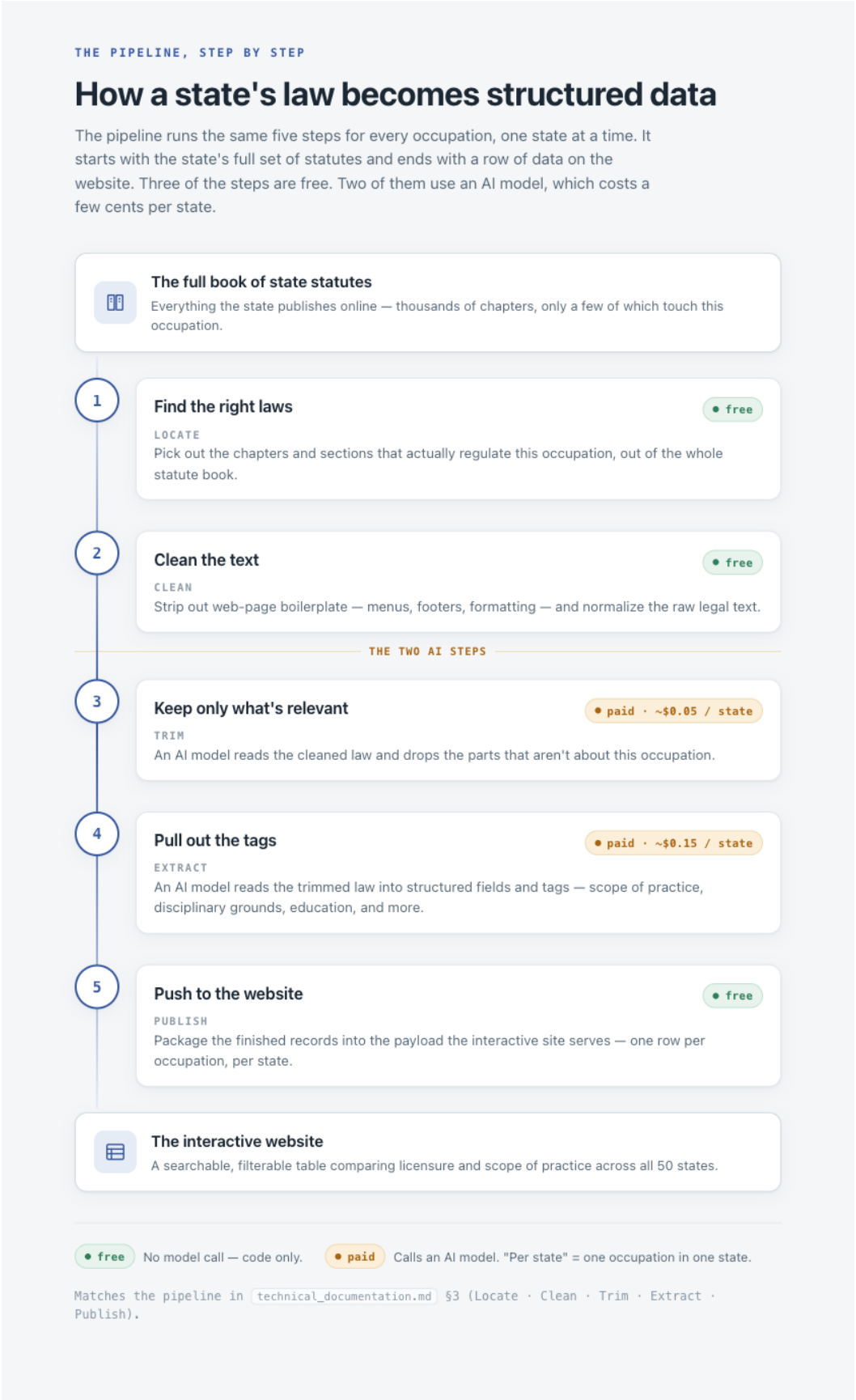


Figure 1: The pipeline, step by step: the full book of state statutes flows down through five steps — Find the right laws (Locate) and Clean the text are free, Keep only what's relevant (Trim) and Pull out the tags (Extract) are the two paid AI steps, and Push to the website (Publish) is free — ending as the interactive website.

— so this number can go *up* as well as down. Read it as "the known remaining work as of this snapshot," not "the last thing that will ever be needed." (We also stopped counting occupations a state simply doesn't license — verified correct-blanks — as pending work, which is why this number is smaller than it used to be.)

In effort terms:

- **Free, minutes:** publish an occupation whose data is already extracted; re-run the deterministic cleanup step.
- **Cheap, no code:** run an already-fetched occupation/state through the two paid AI steps (about $0.20 per occupation/state cell). "Already fetched" means the raw statute text is saved in this repository — the AI steps just haven't been paid for yet. The dashboard (section 4) shows exactly which cells these are and quotes the cost before anything is spent.
- **Medium: bring a brand-new occupation online for the first time.** Before we can extract structured data for an occupation, an AI step has to figure out *what fields matter for it* — a nurse's data looks different from a plumber's (different requirements, different terminology) — and save that as a one-time template (called a "schema" in the code) for the occupation. This is a one-time step per occupation, not per state, and it's a small cost. This is the 0-cell row above: the law is already on disk, but nobody has designed that occupation's template yet. Once it exists, every state for that occupation becomes "ready to process" like any other. Some states may also still need a fetch.
- **Automated, unattended:** fetching new or updated statute text from the web (section 2). This used to be the hard, babysit-a-browser step; it now runs headless on its own — you kick it off and let it run.

4. The tools you'll actually use

What you need: Python (the repo ships a ready environment — see `pipeline/client/README.md`), an Anthropic API key for the two paid steps, and access to the site's hosting (Vercel) for publishing. Fetching now runs headless — no browser to launch or babysit.

Three commands cover everything except fetching (run from the repo root, using the project's Python environment):

1. **Status** — `python -m pipeline.client.status` — recomputes, for every occupation and state, what stage it's at, whether it's current or stale, and what it would cost to finish.
2. **Dashboard** — `python -m pipeline.client.dashboard` — builds a web page (`pipeline/results/status/`) where you tick the occupations you want, see the total cost, and get the exact commands to run. Running this command *also mirrors the same page to the public site at `/dashboard`* (utah-licensing.vercel.app/dashboard) — so **"the dashboard on the website" and this one are the same page**. The only difference is freshness: the website copy is a **frozen snapshot** from the last time someone ran this command and redeployed, while running it locally regenerates it with today's numbers. Either way it's a *read-only picker* — it shows what's been processed and the commands to run; it never runs anything or spends money itself. (This is separate from the main site, which is the finished licensing-data viewer for the public.)
3. **Runner** — `python -m pipeline.client.run …` — executes your picks: `clean` (free), `trim` and `extract` (the paid steps — each prints its cost estimate and waits for a typed "yes" before spending anything), `publish` (free).

Nothing in these tools fetches from the web or spends money on its own. The plain-English walkthrough, including re-fetching, is `docs/runbooks/client_refetch.md`.

5. Everything adjustable lives in config files, not code

The pipeline's decisions — which occupations exist, how to recognize each one in a state's law, per-state quirks, what the AI is asked, rate limits — are all plain-text YAML files in `pipeline/config/`, each documented in `pipeline/config/README.md`:

- `professions.yaml` — the occupation list and the patterns used to find each one.
- `state_structure.yaml` — per-state source and layout overrides (the single source of truth for "where does this state keep this law").
- `prompts.yaml` — the instructions given to the AI at each paid step.
- `models.yaml` — which AI model each paid stage uses (swap models or vendors here).
- `limits.yaml` — every numeric knob (fetch delays, caps, thresholds).

A non-programmer can read these; a technical person can change them without touching Python.

6. What it costs — and what the costs depend on

Three stages call an AI model and cost money; everything else is free:

Stage	Model used today	Est. cost
Trim — keep only the occupation's parts of the law	`claude-haiku-4-5-20251001`	~$0.05 per state
Extract — turn trimmed text into structured data	`claude-sonnet-4-6`	~$0.15 per state
Schema discovery — design an occupation's data template	`claude-sonnet-4-6`	~$1.00 per occupation, **one-time** (not per state)

"Per state" means one occupation in one state — a single cell of the grid in section 3. So the $0.05 and $0.15 above are the cost to trim / extract *one* occupation's law for *one* state. Doing an occupation across all 50 states runs each paid step 50 times: roughly $0.20 × 50 $10 for that occupation. Schema discovery is the exception — it's paid once per occupation, not per state.

Right now, 0 occupations have law text on disk and are waiting only on this one-time step — discovering all of them would cost about $0.00 total.

Three things to understand about these numbers:

- **They are estimates tied to today's models and prices.** The model names above were read live from `pipeline/config/models.yaml` at generation time. Anthropic's per-token prices change, and bulk runs go through Anthropic's discounted batch API. Treat the per-cell figures as planning estimates, not invoices — the runner shows the estimate and asks before spending.

- **Swapping models — and even switching AI vendors — is a config edit, not a code change.** Each stage's model lives in one plain-text file, `pipeline/config/models.yaml` (keys `trim`, `extract`, `schema`), with a `--model` flag to override a single run. The trim and schema stages call models through a **vendor-neutral library (LiteLLM)**, so to move one from Claude (Haiku/Sonnet) to, say, OpenAI's ChatGPT you put its model name in `models.yaml` (e.g. `gpt-5`) and set that vendor's API key in the environment — nothing else.
- **Extract has two backends, and you choose per run — this is the "bulk" trade-off.** The **default** extract path sends the whole job through Anthropic's **Message Batches API**, which gives a ~**50% bulk discount** (plus prompt-cache reuse) — this is what keeps big re-runs cheap, and it is Claude-only. The alternative, `... run extract --backend litellm`, runs each cell one-at-a-time through LiteLLM, so **any vendor works** (`--model gpt-5`, a Gemini model, etc.) — at the cost of the bulk discount (budget roughly 2× per cell). So extracting with ChatGPT is `run extract --backend litellm --model gpt-5` plus an OpenAI key — still no code change. In every case the prompts live separately in `pipeline/config/prompts.yaml`, so the model and the prompt change independently.
- **The costs are per-run, not per-view.** The website serves previously-built data; people browsing the site cost nothing.

7. How much to trust the data

- **It's AI-extracted, with checks.** Structured records are extracted by an AI model from statute text, then run through automated consistency checks (`pipeline/eval/check_consistency.py`). It can still be wrong; every record links back to its source law so a human can verify.
- **"Done" doesn't mean "done forever."** Each cell's output depends on its inputs (the law text) and its recipe (prompt + schema + model + code). The repo stamps every output and can tell you whether it's *current*, *stale* (an input or the recipe changed since it was built), or *unstamped* (built before tracking existed). Right now: 1499 extracted cells — 1126 current, 0 stale, 373 unstamped. The dashboard color-codes this.
- **Some blanks are deliberate.** A hand-verified list (`pipeline/config/verified_absences.yaml`) records occupation/state pairs where the state genuinely has no license — shown as "not state-regulated (verified)," distinct from "we haven't processed this yet."
- **Having a chapter isn't the same as having every relevant law.** Some states put discipline, fees, or hearing rules in shared "umbrella" chapters that apply to many occupations; the repo detects and pulls these, but completeness auditing is ongoing.
- **We actively hunt for our own mistakes, and they do happen.** Two purpose-built audits exist because processed-and-published is not the same as verified-correct: `pipeline/discover/audit_corpus.py` re-checks whether a pulled corpus is actually that occupation's licensure law — on its validation run it correctly flagged the real mistakes (a repealed statute counted as current law, a fire-safety or criminal statute pulled instead of the licensing act) with no false alarms. `pipeline/eval/corpus_drift.py` catches a different failure: a corpus that silently stopped matching its own mapping after the mapping changed (a real bug it found: a state's committed corpus was still answering to a mapping that had been replaced, and nobody had noticed because the page still "had text" on it). Both exist precisely because a wrong-but-confident-looking record is possible — running them, and reading what they flag, is the honest way to answer "how much can we trust this," not the 1126-current/0-stale/373-unstamped counts above on their own.

8. How this was built (Claude Code)

Almost all of this software — the web scraping, the cleaning and extraction steps, the config system, the status/dashboard tools, even the generator that writes this document — was built using **Claude Code**, Anthropic's AI coding assistant, working alongside a human. In practice that meant a person describing what was needed in plain English ("find the chapter that regulates dietitians in each state," "flag any pulled law that stopped matching its mapping") and Claude Code writing, running, and debugging the actual Python, with the human reviewing and steering. The AI did most of the typing and much of the exploration; the human made the decisions and checked the results.

Two things worth knowing about that for whoever inherits this:

- **The same approach is available to you.** You do not have to be a Python expert to extend or fix this pipeline. Because the moving parts live in plain-text config files (section 5) and small, well-separated modules, you can describe a change in plain English to Claude Code (or a similar assistant) and have it make the edit — the way the pipeline was built in the first place.
- **This is different from the AI *inside* the pipeline.** Claude Code built the machine; the models in section 6 (Haiku, Sonnet) are parts *inside* the machine that read statutes every time it runs. Building it was a one-time human-plus-AI effort; those runtime models are what cost money per state each time you process new law.

Regenerate with `python -m pipeline.client.handoff_doc [--pdf]`.